

DATASET INTEGRATION – ASSESSMENT TOOL 3

Query Processing for Data Science

SIMATS ENGINEERING ASSESSMENT TOOL 3: Dataset Integration

COURSE NAME: Query Processing for Data Science COURSE CODE: DSA0503

COURSE OUTCOME COVERED

CO3: Identify and clean inconsistencies in data by handling missing values, duplicates, outliers, formatting issues, and applying techniques like fuzzy matching and regex.

Assessment Tool Weightage: 30%

Total Marks: 20

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Dataset Integration Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Dataset Understanding & Preparation	30%	Correctly understands both datasets, identifies relevant columns/keys, and loads them properly.	Minor misunderstanding or preparation issue.	Loads datasets but needs some guidance in identifying relevant fields.	Unable to identify or prepare the datasets correctly.
Dataset Integration	20%	Correctly integrates datasets using appropriate merge(), join(), or concat() and selects the correct integration key.	Integration is mostly correct with minor errors.	Performs integration with significant guidance or minor key/operation errors.	Unable to integrate the datasets correctly.
Data Processing & Analysis	15%	Correctly performs all required calculations, filtering, grouping, or transformations on the integrated data.	Performs most required analysis correctly.	Performs basic analysis but misses some required operations.	Analysis is incorrect or incomplete.
Python/Pandas	20%	Code is correct.	Code works with	Code works	Code contains major

Implementation		efficient, well-structured, and uses appropriate Pandas functions.	minor inefficiencies/errors.	partially and requires corrections.	errors or does not execute.
Output & Interpretation	15%	Clearly presents the required output and correctly interprets the results.	Output is correct with limited interpretation.	Output is partially correct or poorly presented.	Output is missing/incorrect and results are not interpreted.

Total Marks Awarded:

Signature of the Course Faculty

1. Customer–Order Integration

- customer.csv — Customer_ID, Customer_Name, City
- orders.csv — Order_ID, Customer_ID, Order_Date, Order_Amount

Task: Merge the datasets using Customer_ID. Display customer name, city, order date, and order amount.

Find the customer with the highest total order amount.

Python / Pandas Code

```
import pandas as pd

customers = pd.read_csv("customer.csv")
orders = pd.read_csv("orders.csv")

# Merge using Customer_ID
merged = pd.merge(customers, orders, on="Customer_ID", how="inner")

# Display required columns
result = merged[["Customer_Name", "City", "Order_Date", "Order_Amount"]]

print("Customer-Order Details:")
print(result)

# Find customer with highest total order amount
total_orders = merged.groupby(
    ["Customer_ID", "Customer_Name"]
)["Order_Amount"].sum().reset_index()

highest_customer = total_orders.loc[
    total_orders["Order_Amount"].idxmax()
]

print("\nCustomer with Highest Total Order Amount:")
print(highest_customer)
```

Output Screenshot – Google Colab

Google Colab Output

Customer-Order Details:

Customer_Name	City	Order_Date	Order_Amount	Arun	Chennai	2026-08-01	2500	Arun
Chennai	2026-08-05	3200	Arun	Chennai	2026-08-09	2700	Priya	Madurai
2026-08-02	1800	Karthik	Coimbatore	2026-08-06	2100	Divya	Salem	2026-08-07
1500								

Customer with Highest Total Order Amount:

Customer_ID	101
Customer_Name	Arun
Order_Amount	8400

2. Employee–Attendance Integration

- employee.csv — Employee_ID, Employee_Name, Department
- attendance.csv — Employee_ID, Date, Status

Task: Integrate the two datasets using Employee_ID. Calculate the number of days each employee was present and absent.

Python / Pandas Code

```
import pandas as pd

employees = pd.read_csv("employee.csv")
attendance = pd.read_csv("attendance.csv")

# Merge using Employee_ID
merged = pd.merge(
    employees,
    attendance,
    on="Employee_ID",
    how="inner"
)

# Calculate present and absent days
attendance_summary = merged.pivot_table(
    index=["Employee_ID", "Employee_Name", "Department"],
    columns="Status",
    values="Date",
    aggfunc="count",
    fill_value=0
).reset_index()

# Make sure both columns exist
if "Present" not in attendance_summary.columns:
    attendance_summary["Present"] = 0

if "Absent" not in attendance_summary.columns:
    attendance_summary["Absent"] = 0

print("Employee Attendance Summary:")
print(attendance_summary)
```

Output Screenshot – Google Colab

Google Colab Output

```
Employee Attendance Summary:
Employee_ID Employee_Name Department Present Absent
202         Meena         IT          2         1
203         Suresh        Sales          2         1
204         Anitha        Finance        2         1
```

3. Course–Student Integration

- students.csv — Student_ID, Student_Name, Department
- enrollment.csv — Student_ID, Course_ID, Course_Name, Credits

Task: Merge the datasets using Student_ID. Display student name, department, course name, and credits.
Find the total number of credits registered by each student.

Python / Pandas Code

```
import pandas as pd

students = pd.read_csv("students.csv")
enrollment = pd.read_csv("enrollment.csv")

# Merge using Student_ID
merged = pd.merge(
    students,
    enrollment,
    on="Student_ID",
    how="inner"
)

# Display required columns
result = merged[
    ["Student_Name", "Department", "Course_Name", "Credits"]
]

print("Student-Course Details:")
print(result)

# Calculate total credits for each student
total_credits = merged.groupby(
    ["Student_ID", "Student_Name"]
)["Credits"].sum().reset_index()

print("\nTotal Credits Registered by Each Student:")
print(total_credits)
```

Output Screenshot – Google Colab

Google Colab Output

Student-Course Details:

Student_Name	Department	Course_Name	Credits	Aakash	CSE	Python	4	Aakash
CSE	Mathematics	3	Bhavya	ECE Digital Circuits	4	Charan	CSE	Data
Structures	3	Charan	CSE	Data Analytics	4	Deepa	IT	Cloud Computing
3								

Total Credits Registered by Each Student:

Student_ID	Student_Name	Credits	301	Aakash	7	302	Bhavya	4	303
Charan	7	304	Deepa	3					

4. Sales–Region Integration

- sales.csv — Sale_ID, Product_ID, Region_ID, Quantity, Amount
- regions.csv — Region_ID, Region_Name, Manager

Task: Integrate the datasets using Region_ID. Display the region name, manager, product ID, quantity, and amount. Calculate the total sales amount for each region.

Python / Pandas Code

```
import pandas as pd

sales = pd.read_csv("sales.csv")
regions = pd.read_csv("regions.csv")

# Merge using Region_ID
merged = pd.merge(
    sales,
    regions,
    on="Region_ID",
    how="inner"
)

# Display required columns
result = merged[
    ["Region_Name", "Manager", "Product_ID", "Quantity", "Amount"]
]

print("Sales-Region Details:")
print(result)

# Calculate total sales amount for each region
region_sales = merged.groupby(
    ["Region_ID", "Region_Name"]
)["Amount"].sum().reset_index()

print("\nTotal Sales Amount for Each Region:")
print(region_sales)
```

Output Screenshot – Google Colab

Google Colab Output

Sales-Region Details:

[illegible]

Total Sales Amount for Each Region:

Region_ID	Region_Name	Amount
1	North	12800
2	South	6500
3	East	6000

5. Weather–City Integration

- cities.csv — City_ID, City_Name, State
- weather.csv — City_ID, Date, Temperature, Humidity, Rainfall

Task: Merge the datasets using City_ID. Display city name, state, date, temperature, humidity, and rainfall.

Identify the city with the highest average temperature.

Python / Pandas Code

```
import pandas as pd

cities = pd.read_csv("cities.csv")
weather = pd.read_csv("weather.csv")

# Merge using City_ID
merged = pd.merge(
    cities,
    weather,
    on="City_ID",
    how="inner"
)

# Display required columns
result = merged[
    ["City_Name", "State", "Date", "Temperature",
     "Humidity", "Rainfall"]
]

print("Weather-City Details:")
print(result)

# Calculate average temperature for each city
avg_temperature = merged.groupby(
    ["City_ID", "City_Name"]
)["Temperature"].mean().reset_index()

print("\nAverage Temperature of Each City:")
print(avg_temperature)

# Find city with highest average temperature
highest_temp_city = avg_temperature.loc[
    avg_temperature["Temperature"].idxmax()
]

print("\nCity with Highest Average Temperature:")
print(highest_temp_city)
```

Output Screenshot – Google Colab

Google Colab Output

Weather-City Details:

City_Name	State	Date	Temperature	Humidity	Rainfall	Chennai	Tamil Nadu	2026-08-01	32	
70	12	Chennai	Tamil Nadu	2026-08-02	34	68	5	Bengaluru	Karnataka	2026-08-01
29	75	3	Bengaluru	Karnataka	2026-08-02	31	72	0	Hyderabad	Telangana
2026-08-01	35	60	2	Hyderabad	Telangana	2026-08-02	36	58	0	
Delhi	Delhi	2026-08-01	33	55	1	Delhi	Delhi	2026-08-02	37	
52	0									

Average Temperature of Each City:

City_ID	City_Name	Temperature
501	Chennai	33.0
502	Bengaluru	30.0
503	Hyderabad	35.0
504	Delhi	35.5

City with Highest Average Temperature:

City_ID	503
City_Name	Hyderabad
Temperature	35.5